

Department of Software Engineering

Choosing a Development Model for the SDLC

Submitted to:

Mr. Adeel Shahzad

Submitted by:

Fiza Noor (25021598-008)

Kashaf (25021598-020)

Rizwan Asif (25021598-051)

Isha Iman (25021598-063)

Amna Malik (25021598-156)

Zamarah Shamim (25021598-200)

Muhammad Aliyan (25021598-245)

Section:

AB

Table of Content

1. The Client's Situation and Needs 1

1.1 About the Client 1

1.2 The Problem 1

1.3 Plan: Five Releases 1

2. Why the Incremental Model Is the Best Choice 2

2.1 Assumed Schedule for the First Two Releases 2

3. Why the Other Models Were Not Chosen 3

3.1 Waterfall Model 3

3.2 V-Model 3

3.3 Agile (Scrum) 3

3.4 Spiral Model 3

3.5 Evolutionary Model 3

4. When Another Model Would Be Better 4

5. Comparison of All Models 5

1. The Client's Situation and Needs

1.1 About the Client

Client: Mr. Asad Ali, Owner of Asad Rent-a-Car
Business: A car rental business with more than 5 vehicles and 3 full-time drivers . Bookings and billing are currently done on paper registers.

1.2 The Problem

- **Short deadline (6 weeks):** The busy season for tourists and business travellers starts in six weeks. The business cannot wait 8 to 10 months for a finished system. It needs something working soon to stop double bookings and lost payment records.
- **Clear and stable requirements:** Car rental works in a well-known way. Vehicle records, customer licence details, rental length, deposits and invoices all follow fixed rules, so the client already knows what he needs.
- **Staff need time to learn:** The staff have limited computer skills. Giving them all seven screens at once would cause confusion. They should first learn to handle customers and check out vehicles, and only later driver assignment and expense tracking.
- **Limited client time:** Mr. Asad runs the business day to day and cannot attend daily meetings or regular review sessions, which Scrum expects. He can, however, review and approve a working release when it is ready.

1.3 Plan: Five Releases

To reduce risk and still meet the deadline, the system is split into five releases (increments). Each one adds new working screens on top of the previous ones:

Release	What it covers	Screens included and what they are for
Increment 1	Vehicles and customers	The basic records: vehicle status and customer details.
Increment 2	Bookings and payments	Creating rentals, calculating rental cost
Increment 3	Drivers	Managing the driver list and assigning drivers to rentals.
Increment 4	Costs and maintenance	Fuel, repairs, workshop jobs and fines.
Increment 5	Management reports	Key figures, how much of the fleet is in use, and profit or loss.

2. Why the Incremental Model Is the Best Choice

The Incremental Model splits the whole system into smaller parts and delivers them one after another. The overall design (database and class structure) is planned once at the start, and each release adds new features on top of it. The requirements are agreed first and do not change from release to release.

- **Value arrives early:** The biggest risks, which are lost invoices and double bookings, are solved before the busy season. The company does not need to wait until month nine for a complete system.
- **Lower risk:** If money or time runs short before Increment 4, the company still has a working system. It never ends up with a project that is 90% coded but cannot be used.
- **Easier for staff:** Staff learn customer and vehicle records first. Drivers, repairs and reports are added later, once they are comfortable.

2.1 Assumed Schedule for the First Two Releases

The following is an assumed plan for illustration, not a measured estimate. A real project would adjust it after estimating the actual effort.

When	What happens
Week 1	Final requirements, database design and class design.
Weeks 2-3	Build and test Vehicle View and Customerm View. Release 1 goes live at the shop..
Weeks 4-5	Build and test Sales View and Payment View.
Week 6	Final testing, staff training, and Release 2 goes live before the busy season.

3. Why the Other Models Were Not Chosen

Each of the other models was compared against business's situation: a short deadline, clear requirements, staff who need time to learn, and a busy client.

3.1 Waterfall Model (one step after another)

- **Nothing to use until the end:** Waterfall finishes requirements, design, coding and testing for the system before anything is delivered. The business would miss the busy season if we work with this model.
- **Problems found late:** Mistakes, such as two clerks booking the same car at once or a screen that is hard to use, would only show up when everything is put together at the end.

3.2 V-Model (each stage has a matching test stage)

- **No early working software:** The V-Model is a model where each design stage is paired with a test stage. It only delivers one complete system at the end. A working booking desk by week 6 is not possible.
- **More paperwork than needed:** Matching every design step to a formal test step is more effort than this fairly simple business application needs.

3.3 Agile (Scrum)

- **Not much to discover:** Scrum is used for projects where requirements keep changing, such as new products. Car rental is well understood, so re-planning every module adds work without much benefit.
- **Too much client time:** Scrum relies on the client being closely involved. Assuming this takes several hours every week, Mr. Asad cannot provide it.

3.4 Spiral Model (repeated rounds of risk checking)

- **Too much checking for a simple system:** The Spiral Model repeats the planning, risk analysis, building and review part. It is useful for large projects. the risk reviews cost more time and money than we save.
- **Low technical risk:** Record-keeping screens connected to a database are well understood, so there is little risk that needs this much analysis.

3.5 Evolutionary Model (build a prototype, improve it with feedback)

- **Assumes the client is unsure what he wants:** This model suits projects where the client cannot describe the requirements and must see something first. Mr. Asad already knows what the business needs.
- **Risk to the database:** Evolutionary Model would require us to keep delivering quick prototypes. This could disruption with the building of a clean and reliable database.

4. When Another Model Would Be Better

No model is best in every case. The table shows what would have to be different about the project for each of the other models to beat the Incremental Model:

MODEL	SITUATION WHERE IT FITS	WHAT THE PROJECT WOULD LOOK LIKE
WATERFALL	Fixed rules and a fixed contract	• The client is a body with strict rules. • The deadline is not early and nothing is gained by delivering early. • All tax, vehicle registration are fully written down and will not change.
V-MODEL	Software that controls vehicle hardware	• A software fault could cause accidents or injury. • Requires formal testing at every level. • The system connects directly to vehicle devices, such as remote ignition lock or tracking.
AGILE (SCRUM)	A new idea with an unproven market	• Building a new car-sharing platform between private owners. • It is not known whether hourly rental or subscriptions will work. • The client pays for a product where priorities change every two weeks.
SPIRAL	Research with unclear technology and risk	• Building image-recognition software that spots scratches on cars. • Each round needs a risk assessment before more money is spent. • It is unclear how accurate it can be or who is responsible for damage.
EVOLUTIONARY	Client cannot describe what he needs	• A members-only club with unusual, non-standard procedures. • The client can only say 'I will know it when I see it.' • Several versions of the screens must be tried before the real needs are clear.

5. Comparison of All Models

Model	Main idea	Fit for Business	When would it be right
Incremental (chosen)	Build and deliver in stages on a stable design	**Best fit**: gives a working booking system within 6 weeks and matches the modular design.	Clear requirements and a firm go-live date.
Waterfall	Finish each phase fully, deliver once at the end	Too slow: misses the busy season and finds problems late.	Fixed requirements and a long deadline, such as a government contract.
V-Model	Every design stage has a matching test stage	More testing paperwork than needed, and still only one delivery at the end.	Software controlling vehicle hardware, where failures are dangerous.
Agile (Scrum)	Short sprints; requirements change as the product develops	Too much re-planning for a problem; needs more of the client's time than he has.	New products with changing requirements and an involved client.
Spiral	Repeated rounds of risk analysis and prototypes	Too much risk review for a simple database application.	Experimental technology or high-risk research.
Evolutionary	Improve a prototype using user feedback	Quick changes could damage the database design that booking records depend on.	Client cannot describe the workflow without seeing prototypes.